

SPACE INVADERS: ASSEMBLY EDITION

Project Guidelines — CS 003 (Computer Architecture & Organization)

Course & Section	CS 003 - CS21S2 (BSCS)	Instructor	Ma'am Monaliza Gregorio
Leader	Ethan Condaya	Estimated Timeline	4 to 5 weeks
Team	Blanco · Condaya · Dequito · Jumawan · Silvero · Velasco		

1. What Are We Building?

We're recreating **Space Invaders** entirely in **x86 assembly (NASM)**, rendered through the Windows GDI, with our own added features layered on top of the classic arcade formula. Nothing here is handed to us by a game engine — every sprite draw, every timer tick, and every score update happens through direct manipulation of registers, memory addresses, and the stack.

This is the same connection our proposal already made: fire-rate cooldowns and the alien speed curve force us to work with timers and clock cycles ourselves, and even the score tracker becomes a lesson in memory addressing and I/O, since we manually move data between memory and the display. This guide turns that proposal into a week-by-week, person-by-person plan.

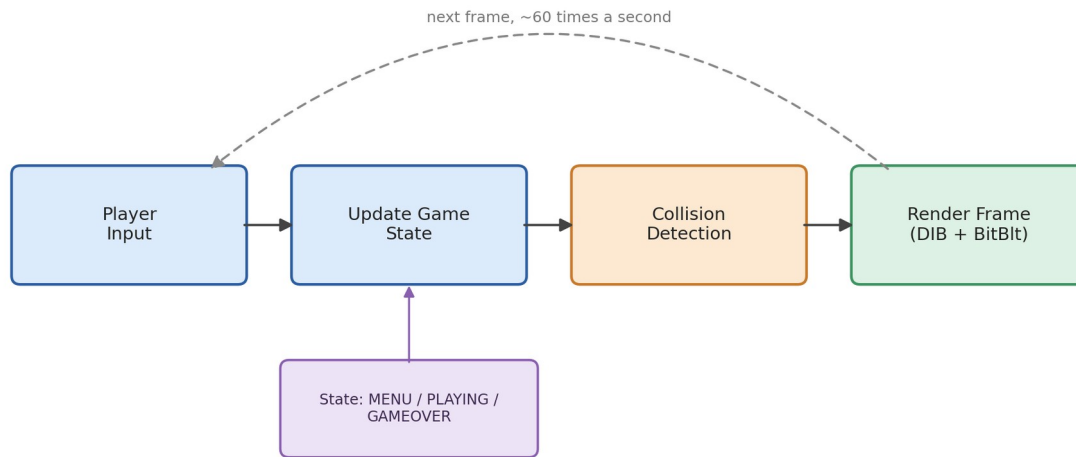
2. Key Terms You'll Run Into

A quick reference — flip back here any time a word doesn't ring a bell.

- **Game Loop** — The repeating cycle at the heart of the program: read input → update state → check collisions → render a frame → repeat, dozens of times a second.
- **State Machine** — The game moving between a small number of distinct modes — MENU, PLAYING, GAMEOVER — where the same key press means something different depending on which mode you're in.
- **DIB Section** — An in-memory pixel buffer (Device-Independent Bitmap) that our code writes to directly, pixel by pixel, before the finished frame is shown on screen.
- **BitBlt** — The single Win32 call that copies our finished pixel buffer onto the visible window in one shot — this is what avoids visible flicker.
- **Calling Convention** — The agreed-upon rule for how arguments get passed into a function — which registers, in what order, who cleans up the stack afterward. We have to follow this exactly to call Win32 API functions from NASM.
- **Handle (HWND)** — A number Windows hands us to identify our window; we pass it back into later API calls whenever we need to do something to that window.
- **Collision Detection (AABB)** — Checking whether two rectangles overlap — bullet vs. alien, alien vs. bunker, and so on. The same basic math scales up to real physics engines.
- **Frame Timer / Delta Time** — Measuring how much time passed since the last frame, so speed-based features (fire cooldown, alien speed curve) behave consistently no matter how fast the loop is running.
- **Data-Driven Design** — Storing each enemy type's stats — point value, sprite, fire pattern — in a table instead of duplicating logic per type. This is how we'll build Multiple Enemy Types without needing OOP classes.

3. How the Game Works, Step by Step

Every frame of the game moves through the same four stages:



The loop reads **Player Input** (keyboard state), **Updates Game State** — player position, alien positions, timers, and which alien speed tier we're on — checks **Collisions** between bullets, aliens, the player, and bunkers, then **Renders** the finished frame through the DIB section and BitBlt. The current state (MENU / PLAYING / GAMEOVER) gates what "Update Game State" actually does each frame — e.g. in MENU, it only listens for Enter; in GAMEOVER, it stops updating aliens entirely.

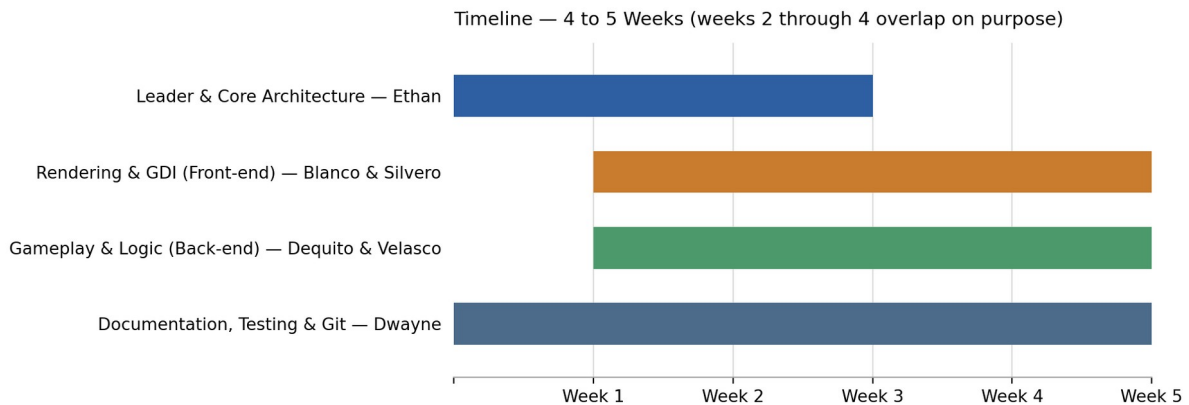
4. Team Roles

Role	Owner(s)	Responsibility
Leader & Core Architecture	Ethan	Owns the game loop skeleton and state machine; decides the memory layout for player/alien/bullet data up front so both teams build against the same interface; runs weekly check-ins; merges and integrates the rendering and gameplay branches; has final say on scope calls (e.g. cutting a stretch goal if the team is behind).
Rendering & Windows/GDI (Front-end)	Blanco, Silvero	Window creation, the DIB section pixel buffer, the BitBlt draw loop, sprite-drawing routines, on-screen score/lives text, particle-effect rendering, and the menu / game-over screen visuals.
Gameplay & Logic (Back-end)	Dequito, Velasco	Player movement and input handling, alien grid movement and the dynamic speed curve, bullet management (player fire and alien return fire), collision detection, bunker damage logic, power-up drop/pickup, the enemy-type stat table, and boss logic (stretch).
Documentation, Testing & Git Hygiene	Dwayne	Written report and requirement deliverables for Ma'am Gregorio, the README, a test-case checklist and bug tracker, and GitHub repo hygiene (issue labels, PR reminders) — freeing the other four to stay heads-down in code.

Roles are primary ownership, not silos. Pair up for review — especially Architecture ↔ Rendering and Architecture ↔ Gameplay — since Ethan's data-structure decisions are what both teams build directly on top of. Everyone can (and should) touch every part of the game when it makes sense; these are default owners, not walls.

5. Timeline (4 to 5 Weeks)

No fixed calendar dates here on purpose — Ethan sets the actual start/end dates for each week. Weeks 2 through 4 overlap intentionally, once Week 1's interfaces are agreed on. Each week below has its own checklist per person/team, so everyone knows exactly what “done” looks like before moving on.



Week 1 — Setup & Foundations

Owner	Checklist
Leader — Ethan	☐ Create the GitHub repo with a branch structure (main + feature branches) and a commit-message convention ☐ Write the initial README (project overview, how to build/run) ☐ Draft the player/alien/bullet memory layout on paper and share it for the team to review ☐ Confirm each team's assigned features for the whole project (see Section 7)
Rendering — Blanco & Silvero	☐ Complete the GitHub Skills “Introduction to GitHub” module ☐ Install NASM + a linker (Microsoft Build Tools or MinGW-w64) and confirm it builds ☐ Build a minimal Win32 “blank window” test to confirm the toolchain works end-to-end ☐ Review Ethan's memory-layout draft and flag anything that affects rendering
Gameplay — Dequito & Velasco	☐ Complete the GitHub Skills “Introduction to GitHub” module ☐ Install NASM + a linker and confirm it builds ☐ Review Ethan's memory-layout draft and flag anything that affects gameplay logic ☐ List out the exact collision pairs needed (bullet–alien, alien–player, alien–bunker, etc.)

Docs & Testing — Dwayne	☐ Complete the GitHub Skills “Introduction to GitHub” module ☐ Set up a shared test-case tracking sheet ☐ Draft the report's intro/concept section (course connection, what the game is) ☐ Set up the GitHub Projects task board for the team
------------------------------------	--

Week 2 — Core Engine + First Draw

Owner	Checklist
Leader — Ethan	☐ Build the game loop skeleton (input → update → collide → render) ☐ Implement the state machine stub (MENU / PLAYING / GAMEOVER) ☐ Review and merge each team's first pull request into main
Rendering — Blanco & Silvero	☐ Create the window and the DIB section pixel buffer ☐ Get the BitBlt draw loop rendering a static frame ☐ Draw a static player sprite and the alien grid
Gameplay — Dequito & Velasco	☐ Implement player movement and input handling ☐ Store alien grid positions in memory ☐ Implement basic bullet firing (no collisions yet)
Docs & Testing — Dwayne	☐ Populate the GitHub Projects board with Week 2–5 tasks ☐ Start the test-case checklist for movement and input ☐ Check in with both teams and log any blockers

Week 3 — Full Loop + Easy Tier + Menu

Owner	Checklist
Leader — Ethan	☐ Wire the full state machine into the loop (menu ↔ playing ↔ game-over transitions) ☐ Review and merge the collision-detection code ☐ Run the first full integration build
Rendering — Blanco & Silvero	☐ Draw the on-screen score & lives text ☐ Build the menu and game-over screens ☐ Add the Mystery UFO sprite and animation
Gameplay — Dequito & Velasco	☐ Implement collision detection (bullet ↔ alien, alien ↔ player/bottom) ☐ Implement the fire-rate cooldown ☐ Implement the dynamic alien speed curve ☐ Implement alien return fire ☐ Implement Mystery UFO logic (timing, movement,

	scoring)
Docs & Testing — Dwayne	☐ Run the first full team playtest ☐ Log every bug found into the tracker ☐ Update the report with progress so far

Week 4 — Remaining Medium + Required Advanced

Owner	Checklist
Leader — Ethan	☐ Confirm the bunker and power-up data structures fit the existing memory layout ☐ Review and merge the Advanced-tier feature branches ☐ Decide whether the team starts a stretch goal this week
Rendering — Blanco & Silvero	☐ Draw destructible bunkers, block by block ☐ Implement the particle-explosion rendering ☐ Draw the 3 enemy-type sprites (Squid/Crab/Octopus) and the power-up icons
Gameplay — Dequito & Velasco	☐ Implement bunker damage logic ☐ Implement particle spawn/expire logic ☐ Build the enemy-type stat table (points, sprite offset, fire pattern) ☐ Implement power-up drop, pickup, and timed effects
Docs & Testing — Dwayne	☐ Run an integration test pass across rendering and gameplay ☐ Update the test-case checklist for every new feature ☐ Continue writing the report's features section

Week 5 — Stretch Goals, Polish & Demo

Owner	Checklist
Leader — Ethan	☐ Decide which stretch goal(s) to attempt based on how stable the game is ☐ Run the final integration and freeze merges ahead of the demo ☐ Coordinate the demo run-through with the whole team
Rendering — Blanco & Silvero	☐ Polish visuals (menu, HUD, animations) based on playtest feedback ☐ If time allows: build the high-score display UI ☐ If time allows: build the boss sprite and health-bar UI
Gameplay — Dequito & Velasco	☐ Fix bugs found during team playtests ☐ If time allows: implement high-score file I/O ☐ If time allows: implement boss AI and attack phases

Docs & Testing — Dwayne	☐ Run the final full playtest and confirm every required feature works ☐ Finalize the README and the written report ☐ Tag a GitHub release ☐ Rehearse the live demo with the team
------------------------------------	--

Running 4 weeks instead of 5? Don't compress by rushing Week 1's GitHub/toolchain setup — that's exactly the step a first-time-Git team can't afford to skip. Cut a stretch goal before you cut Week 5's polish and testing time.

6. Tools & Platforms

Need	Tool	Why
Assembler	NASM	Free, cross-platform — the assembler the team has chosen.
Linker	Microsoft Build Tools (link.exe) or MinGW-w64 (ld)	Turns NASM's .obj output into a runnable .exe linked against kernel32 / user32 / gdi32.
Graphics & window	Windows API + GDI (DIB section + BitBlt)	No extra install — every Windows machine already has these DLLs, which avoids SDL2/Allegro setup friction across six different laptops.
Debugger	x64dbg or Visual Studio's debugger	Assembly bugs are far harder to reason about blind — a register/memory view is close to non-negotiable at this scope.
Version control + board	GitHub + GitHub Projects	Free; Projects adds a Kanban board directly inside the same repo.
Learning Git/GitHub	GitHub Skills · Learn Git Branching	Free and interactive — better for a first-time team than a passive video, and this is where Week 1's onboarding time should go.
Team chat	Discord	Free voice + text, easy for a 6-person team.
Shared report doc	Google Docs	Free real-time co-editing for the written report Dwayne is coordinating.
Diagrams	draw.io / diagrams.net	Free, no account needed, good for any report diagrams.

7. Feature Set

Locked in for this project — Easy and Medium tiers in full, plus two required Advanced features, with the remaining two as stretch goals.

Required — Easy Tier

- **Dynamic Speed Curve** — Aliens move faster as fewer remain — driven by a counter checked each time an alien dies.
- **On-Screen Score & Lives Tracker** — Draw "Score: X" and "Lives: Y" each frame using a simple bitmap font blitted into the DIB buffer.
- **Player Fire Rate Cooldown** — A frame counter blocks a new shot until roughly half a second has passed since the last one.
- **The Mystery UFO** — A rare bonus alien that streaks across the top of the screen on a timer, worth bonus points if hit.

Required — Medium Tier

- **Alien Return Fire** — Every few frames, pick a random living alien from the data table and spawn a downward-moving enemy shot.
- **Destructible Bunkers** — Each bunker is a small grid of blocks in memory; a hit clears the block it touches instead of the whole bunker.
- **Main Menu & Game Over States** — The state machine from Section 3 — Enter on the menu flips PLAYING on; losing all lives flips GAMEOVER on.
- **Particle Explosion Effects** — On alien death, spawn 5–10 small squares with a random velocity and a short countdown timer before removal.

Required — Advanced Tier

- **Multiple Enemy Types** — A small data table (points, sprite offset, fire pattern) per type — Squid / Crab / Octopus — instead of separate code paths per alien.
- **Dropped Weapon Power-Ups** — A destroyed alien has a chance to spawn a falling icon; colliding with the player applies its effect (shield, speed, spread-shot) for a timed duration.

Stretch Goals (attempt only once the required set is stable)

- **High Score Saving (File I/O)** — Stretch — comparatively the lighter of the two. Write the score to a small file on close, read it back on launch, so the high score survives between sessions.
- **Multi-Stage Boss Fight** — Stretch — the heaviest lift on this list. A boss with a health bar, 20 hits to destroy, and distinct attack phases. Only start this once the required game is stable — it depends on collision, timers, and the enemy-fire pattern all already working.

8. A Few Notes From Your Consultant

- **Small, frequent commits.** One feature = one branch = one PR reviewed by at least one teammate before it merges into main. Since nobody's used GitHub before, treat Week 1's practice time as a real task, not an optional extra.
- **Budget real hours for assembly debugging.** It eats far more time than the equivalent bug would in a high-level language — plan for it rather than treating it as "if we have time."
- **Agree on "destroyed object" representation early.** Bunkers, particles, and power-ups all involve something being destroyed or dropped — if Rendering and Gameplay agree on how that's represented in memory up front, nobody duplicates the same logic twice.
- **If Week 4 slips, cut a stretch goal before cutting Week 5.** A smaller, finished, well-tested game beats a bigger, broken one on demo day.